AMRITA VISHWA VIDYAPEETHAM

School of Computing, Coimbatore Campus

Department of Computer Science & Engineering (AI)

25AIC114 — Elements of Computing II (Assembly Programming)

End-Semester Project Report

PyScope

A Program Execution & Resource Monitoring System

with Cross-Language Comparison and an Interactive IDE

GROUP 9

Name	Roll Number
Dhiren Vasudeva	CB.AI.U4CPS25014
Annareddy Pardhav Reddy	CB.AI.U4CPS25059
Poovya S	CB.AI.U4CPS25061
Ch. Venkata Abhiram	CB.AI.U4CPS25012
Edwin Santosh	CB.AI.U4CPS25045

Submitted to the Faculty of 25AIC114 — Elements of Computing II
Academic Year 2025–2026 | Even Semester

Abstract

Understanding how programs execute internally and how system resources are consumed during execution is a foundational learning objective in the Elements of Computing course. While instruction execution, control flow, and memory usage are taught theoretically, the actual runtime behaviour of a program remains opaque to the learner. PyScope is a practical attempt to close that gap. It is a desktop-grade web application that traces the execution of a user's program line by line, samples CPU, RAM, GPU and step-count telemetry in real time, and renders the results inside an IDE-like interface. The system is built primarily for Python using CPython's built-in sys.settrace mechanism, and is extended to C, C++, Java and JavaScript through a unified subprocess-based runner so that algorithms written in different languages can be benchmarked side by side.

The system is organised as six co-operating modules — a User Program module, an Execution Tracing Engine, a Resource Monitoring module, a Data Collection module, a Visualization module and an Output Display module — exposed through a Streamlit-based premium interface with a dark/light theme. Telemetry collected at each line of execution is buffered in memory and surfaced to the user through four time-series charts (memory, CPU, GPU, step count), a call-stack viewer, a variable timeline, and a comparison view that places two programs head to head. A lightweight, rule-based Insights Engine inspects the captured run and emits human-readable observations such as "tight loop dominates CPU time" or "heap grew steadily — possible accumulation in a list". The application is also packaged as a single Windows executable (PyInstaller) so that it can be demonstrated on machines without a Python installation.

This report documents the motivation, the literature surveyed, the system architecture, the implementation of every module, the results obtained on representative workloads, the limitations encountered, and the avenues for future work. The deliverable is intended both as a learning aid for students who want to see what the interpreter is doing on their behalf, and as a small but realistic demonstration of how concepts taught in Elements of Computing — instruction execution, control flow, the run-time stack and the memory hierarchy — manifest in a working software system.

Keywords — program execution, runtime monitoring, sys.settrace, tracemalloc, psutil, profiling, cross-language benchmarking, Streamlit, educational tooling.

Acknowledgement

We express our sincere gratitude to our course instructor for the course 25AIC114 — Elements of Computing II — for the continual guidance, feedback and encouragement that shaped this project from a small command-line script into the full-featured PyScope application. The structured progression of the course, from the abstract through the mid-review and into the final implementation, gave us repeated opportunities to refine our ideas and our code.

We also thank the Department of Computer Science & Engineering (AI), School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, for providing us with the laboratory facilities and the academic environment needed to carry out this work. Finally, we thank our peers and seniors who patiently tested early builds of the tool, broke it in creative ways, and helped us discover edge cases we would otherwise have shipped with.

— Group 9

Table of Contents

(Tip — right-click and choose "Update Field" in Microsoft Word to refresh page numbers.)

Abstract 2
Acknowledgement 3
Table of Contents 4
1. Introduction 6
1.1 Background 6
1.2 Motivation 6
1.3 Problem Statement 6
1.4 Scope of the Project 7
1.5 Objectives of the Project 7
1.6 Organisation of the Report 7
2. Literature Review 9
2.1 Program-Execution Visualisation 9
2.2 Runtime Resource Monitoring 10
2.3 Gap Analysis 10
3. Tools and Technologies Used 11
3.1 Programming Language 11
3.2 Core Python Libraries 11
3.3 External Compilers and Runtimes 12
3.4 Packaging and Distribution 12
3.5 Development Environment 12
4. System Architecture 13
4.1 Block Diagram 13
4.2 Module Responsibilities 13
4.3 Data-Flow Diagram 14
4.4 Design Decisions 14
5. Methodology and Implementation 16
5.1 Execution Tracing Engine 17
5.2 Resource Monitoring Module 18
5.3 Cross-Language Runner 19
5.4 Visualisation Module 19
5.5 Insights Engine 20
5.6 Sequence of a Single Run 20

5.7 Packaging 21
6. Results and Discussion 22
6.1 Functional Outcomes 22
6.2 Sample Trace — calculate_sum(5) 22
6.3 Comparative Benchmarks 23
6.4 Screenshots 23
6.5 Discussion 24
7. Limitations and Future Work 25
7.1 Known Limitations 25
7.2 Avenues for Future Work 25
8. Individual Contributions 26
9. Conclusion 27
10. References 28
Appendix A — Project Repository Structure 29
Appendix B — Sample User Program 29
Appendix C — Running PyScope Locally 29

1. Introduction

1.1 Background

Modern programming education is unusual in that the act of running a program is the central skill being taught, yet that act is also the one part of the process that students cannot directly see. A learner who writes a `for` loop in Python, an `if` statement in C, or a recursive function in Java sees only two things — the source code on the left of the screen, and whatever output happens to land in a terminal on the right. Everything in between — the bytecode dispatch, the stack frame creation, the heap allocations, the CPU cycles spent — is hidden from them. Instructors compensate by drawing diagrams on the board, by stepping through code in a debugger, or by running profilers, but each of those tools tends to expose only one slice of the execution at a time.

The Elements of Computing II course bridges this gap on the conceptual side by teaching assembly programming and the underlying execution model. PyScope is the practical complement to that: an interactive environment in which a student can paste a program, run it, and watch — in the same window — the line currently executing, the values of the live variables, the depth of the call stack, the amount of memory the process is using and the percentage of the CPU it is consuming.

1.2 Motivation

The motivation for the project came from three observations made during the first half of the course. First, students who could correctly explain in writing how a function call modifies the stack often produced incorrect drawings of the same situation when asked to do it dynamically — they understood frames but not their lifecycles. Second, the existing tools available for free (pdb, cProfile, the Python tutor at pythontutor.com, IDE debuggers) each cover one dimension of the execution but require students to mentally fuse them. Third, debugging a slow program in an undergraduate setting is frequently a matter of guesswork, because students rarely have the experience to know what 'normal' looks like for a memory or CPU graph.

PyScope was therefore designed around three principles: surface as much execution telemetry as possible in a single view; keep the interface aesthetically pleasing so that students will actually want to look at the data; and reduce the cognitive distance between the source code and the resource graphs so that a spike in memory can be visually traced back to the line that caused it.

1.3 Problem Statement

During program execution, four pieces of information are difficult to observe simultaneously: the order in which instructions are executed, the function calls and returns, the values held by live variables, and the CPU and memory consumed. A unified system is required that captures all four during the runtime of a

program, presents them as time-correlated data, and lets the user compare different programs or different implementations of the same algorithm. The system must work without requiring the user to instrument their own code.

1.4 Scope of the Project

PyScope is intentionally not an operating system, a hardware emulator or a production-grade profiler. Its scope is the educational analysis of small to medium programs, typically the kind of code a student might write for a course assignment. Within that scope, the tool aims to be exhaustive: every executed line is recorded, every variable change is tracked, every chart is exportable, and the entire UI is consistent across light and dark themes. Languages outside Python receive a coarser treatment — they are run as subprocesses and only summary statistics (total time, peak memory, CPU%) are captured, because language-specific tracers are out of scope for a single-semester project.

1.5 Objectives of the Project

- Monitor program execution step by step, recording each line that the interpreter executes.
- Track function calls, returns and the call stack depth as they evolve at runtime.
- Observe and persist variable value changes so that a 'variable timeline' can be drawn.
- Measure CPU usage, RAM usage, GPU usage and execution time and align them with execution events on a common time axis.
- Present this telemetry through an interactive, IDE-style web interface that supports both dark and light themes.
- Support a head-to-head comparison of two programs and produce both numerical metrics and a grouped bar chart.
- Generate human-readable insights summarising the run.
- Package the entire application as a single executable so that it can be demonstrated on machines without a Python installation.

1.6 Organisation of the Report

The remainder of this report is organised as follows. Section 2 reviews the prior work in program-execution visualisation and resource monitoring. Section 3 lists the tools and technologies adopted in the implementation, together with the rationale for each choice. Section 4 presents the system architecture, including the modular block diagram and the data-flow diagram. Section 5 walks through the implementation of every module, with code excerpts and the diagrams that explain how the trace hook interacts with CPython. Section 6 documents the experimental results — the screenshots produced by the running application and the comparative benchmarks. Section 7 lists the limitations and the avenues for

future work. Section 8 concludes the report. References and an appendix containing key source listings follow.

2. Literature Review

The work surveyed in this section is split into two streams: program execution visualisation, which is concerned with showing what a program is doing, and resource monitoring, which is concerned with how much of the machine the program is using. The two streams are usually pursued separately. PyScope's contribution is to unify them inside a single, modern web interface.

2.1 Program-Execution Visualisation

S. No.	Paper / System	Main Idea	Limitation Addressed by PyScope
1	CrossCode: Multi-Level Visualisation of Program Execution (2023)	Presents program execution at several abstraction levels — source, intermediate, and a behavioural view — to help learners understand what each construct is doing.	Focuses on visualisation only; provides no insight into CPU, memory or GPU consumption during execution.
2	Educational Program Visualisation Using Execution Information (2024)	Uses execution traces to animate program operations such as array reads and writes inside an interactive web frontend.	Designed mainly for educational data-structure walk-throughs; does not measure runtime resources or compare languages.
3	Graph-Based Program Visualisation (Cal Poly Thesis, 2021)	Represents control flow and inter-function relationships as a directed graph and animates traversal at runtime.	Excellent for static and dynamic structure but does not surface live performance metrics.
4	Software Visualisation: A Survey (Stony Brook Technical Report)	Theoretical survey of techniques for visualising software structure, evolution and execution.	Mostly a taxonomy with limited working implementations or tooling that students can run.
5	Program Visualisation in Computer Science Education (ACM SIGCSE)	Demonstrates that animated visualisations significantly improve student comprehension of programming concepts.	Pedagogical evaluation rather than a tool; PyScope inherits the conclusion and builds the tool.
6	Python Tutor (Guo, 2013)	Web-based tool that lets the user step through Python (and a few other languages) and inspect frame and heap state visually.	Excellent stepping experience but no resource graphs and no comparison view; cannot run on the user's own machine.

Table 2.1 — Comparative summary of related work in program-execution visualisation.

2.2 Runtime Resource Monitoring

Outside the educational sphere, runtime resource monitoring is well-established in production tooling. The Python ecosystem alone provides cProfile, line_profiler, memory_profiler, py-spy, scalene and pyinstrument, each of which collects one or more dimensions of runtime telemetry. At a lower level, the operating system already exports detailed counters via the /proc filesystem on Linux, the Windows Performance Counter API and the Mach task_info API on macOS; psutil offers a consistent abstraction over these. GPUtil, nvidia-smi and the WMI Win32_PerfFormattedData_GPUPerformanceCounters_GPUEngine class expose GPU utilisation. PyScope reuses these mature libraries rather than reinventing them, and concentrates its own effort on correlation and presentation.

2.3 Gap Analysis

The combined picture from the two streams is that no single freely available tool, particularly none aimed at undergraduates, simultaneously offers (a) line-by-line tracing, (b) live CPU / memory / GPU sampling, (c) a polished, themable IDE-style UI, (d) head-to-head program comparison and (e) a packaged distribution. PyScope occupies that gap.

3. Tools and Technologies Used

The technology stack was assembled with two goals in mind: every piece must be free and cross-platform, and every piece must be familiar enough to a second-year undergraduate that the project remains pedagogically transparent.

3.1 Programming Language

- Python 3.9+ — used throughout the application as the implementation language. Python was chosen for its first-class introspection facilities (sys.settrace, tracemalloc, gc, inspect) which make line-level tracing trivial to implement.

3.2 Core Python Libraries

Library	Used For	Why It Was Chosen
sys	Trace hook installation via sys.settrace().	Standard library, no external dependency, native CPython support.
os, time, subprocess	File location, timing, multi-language process spawning.	Standard library; stable cross-platform behaviour.
tracemalloc	Heap allocation tracking; current and peak memory consumed by Python objects.	Reports allocations at line granularity with no third-party install.
psutil	Process-level CPU%, RSS, virtual memory and per-thread sampling.	Mature, single API across Windows, Linux and macOS.
GPUtil + WMI	GPU utilisation; GPUtil for NVIDIA, WMI fallback for AMD/Intel on Windows.	Together they cover the GPU vendors most commonly seen on student laptops.
matplotlib	Static rendering of overview, single-metric, comparison and summary charts.	De-facto standard scientific plotting library; fine-grained control over theme.
numpy	Statistical aggregation (mean, min, max, argmin, argmax) used in the single-metric chart annotations.	Negligible startup cost; vectorised.
streamlit	Frontend framework providing the IDE-style interface, theme switching, and component layout.	Pure Python — keeps the codebase homogeneous and small.

Table 3.1 — Python libraries used and the rôle each plays.

3.3 External Compilers and Runtimes

Cross-language execution is handled by spawning the appropriate compiler or interpreter as a child process. The system requires the relevant toolchain to be present on the user's PATH; missing toolchains are detected at startup and reported through the UI rather than crashing the application.

Language	Toolchain	Invocation Pattern
C	gcc	Compile to a temporary executable, then time and exec it under psutil.
C++	g++	Same pattern as C, with -std=c++17 by default.
Java	javac + java	Compile to .class, run via java with the -Xss option for deep recursion tests.
JavaScript	Node.js (>= 16)	Direct invocation of node user_program.js.

Table 3.2 — External toolchains detected and used by the multi-language runner.

3.4 Packaging and Distribution

- PyInstaller — compiles the entire Streamlit application, its dependencies and the matplotlib backend into a single Windows executable (pyscope.exe). The bundled .spec file controls the included data files, hidden imports (e.g. streamlit.runtime.scriptrunner.magic_funcs) and the runtime hooks.
- A small launcher.py script starts the Streamlit server on a free port and opens a default browser, so that the .exe behaves like a desktop app.

3.5 Development Environment

- Cursor / Visual Studio Code — primary editor, with Pylance and Black for static analysis and formatting.
- Git + GitHub — version control and collaborative development across the five-member team.
- Windows 11, Ubuntu 22.04 — primary test platforms; macOS 14 was used for spot-checks.

4. System Architecture

PyScope is built as six independent modules that communicate through well-defined function-level interfaces. The modular split serves two purposes: it isolates concerns (so the trace hook does not know how a chart is rendered) and it makes it possible to extend the system to additional languages or additional metrics without rewriting the rest of the application.

4.1 Block Diagram

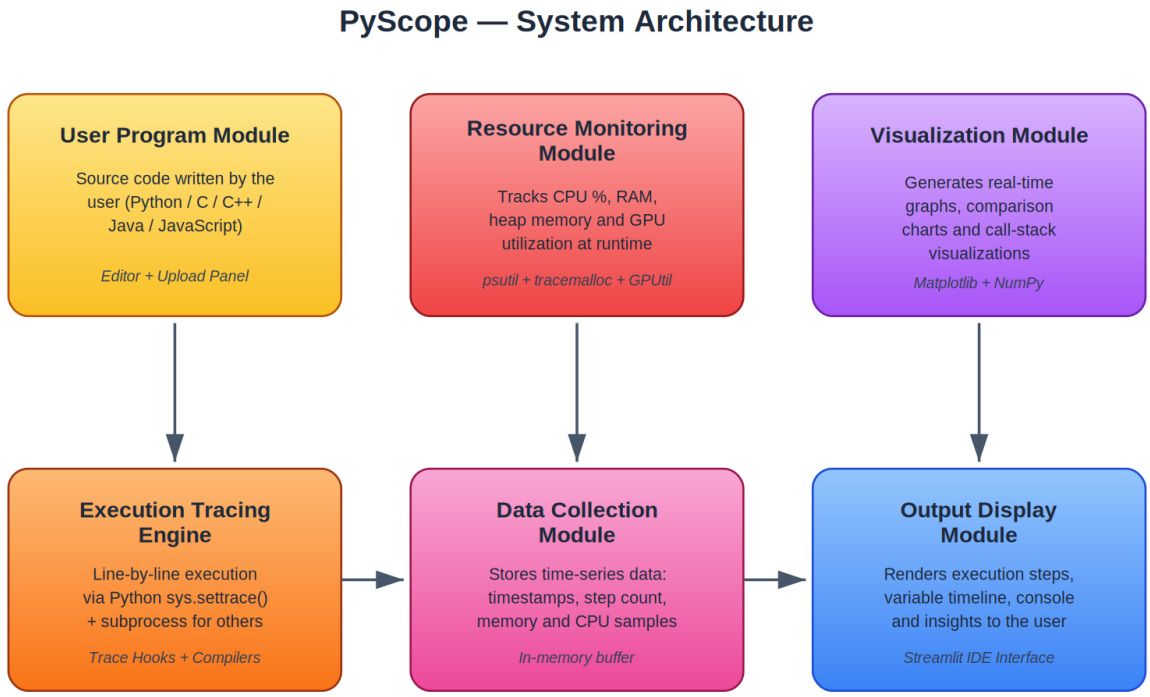


Figure 4.1 — Modular Block Diagram of the PyScope Program Execution and Resource Monitoring System

Figure 4.1 — Modular block diagram of PyScope. Arrows indicate the direction of data flow at runtime.

4.2 Module Responsibilities

Module	Implementation File	Responsibilities
User Program Module	app.py (editor area), user_program.py	Hosts the source code the user types or uploads; serialises it to disk before each run.
Execution Tracing Engine	execution_engine.py, main_100.py	Installs the trace hook, classifies events (call / line / return), records frame and locals.
Resource Monitoring Module	execution_engine.py, multi_lang_runner.py	Samples CPU%, RSS, peak heap, GPU% on every traced line or polling tick.

Module	Implementation File	Responsibilities
Data Collection Module	in-memory buffers in execution_engine.py	Aggregates the time series and presents a single stats dictionary to the visualiser.
Visualization Module	graph_utils.py	Renders the four overview charts, single-metric pop-ups, comparison bar chart, and summary chart.
Output Display Module	app.py, ui_components.py, ui_styles.py	Streamlit components: code editor, console, side panels, theme toggle, navigation.
Insights Engine	insights.py	Inspects a finished run and emits human-readable observations and optimisation hints.

Table 4.1 — Each PyScope module mapped to its source file(s) and its responsibilities.

4.3 Data-Flow Diagram

The level-1 data-flow diagram below captures the way user input is transformed into the final graphical output. The User entity is external; everything inside the dashed boundary is a process or a data store inside PyScope.

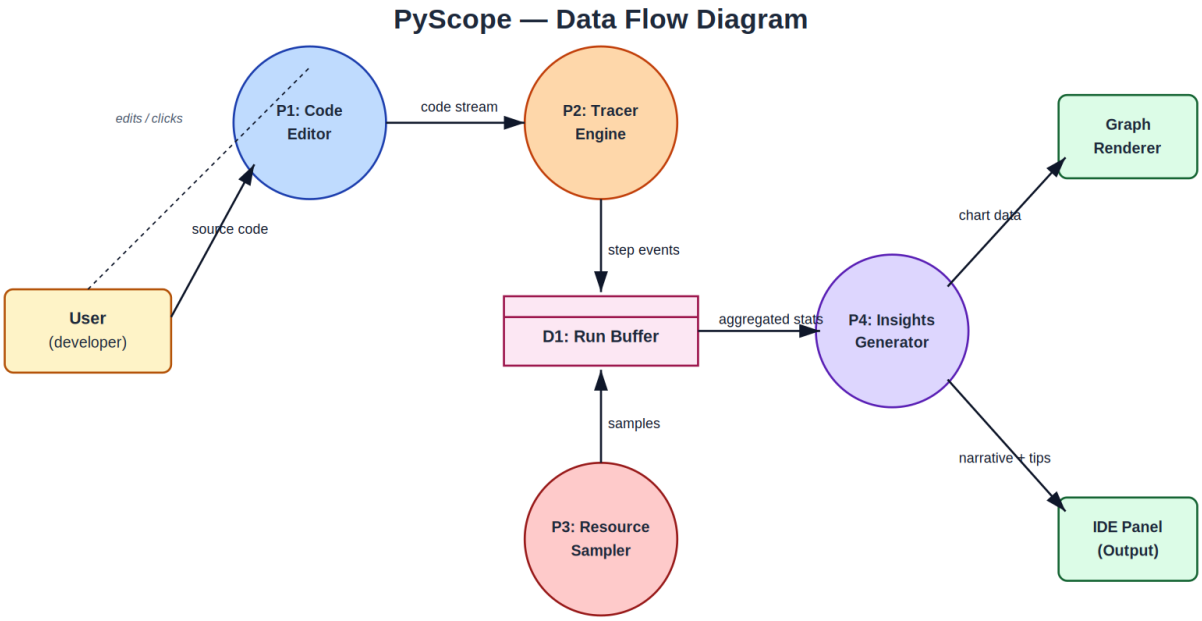


Figure 4.2 — Level-1 Data Flow Diagram showing how user input is transformed into insights

Figure 4.2 — Level-1 DFD showing how source code flows through the trace engine and the resource sampler before being aggregated and presented.

4.4 Design Decisions

- **Python-first, multi-language second.** The Python tracer produces fine-grained data because CPython exposes a built-in trace hook. Other languages do not, so we deliberately settle for coarse summary statistics rather than try to instrument them.
- **Static charts over live charts.** Streamlit re-renders the entire page on every interaction, so a streaming chart would be visually noisy. The trace records a complete time series and renders it once at the end. The user perceives this as instantaneous on programs of educational size.
- **In-memory buffers, no database.** Each run is independent. There is no need to persist data across runs, so a database would be overkill and would expand the deployment footprint.
- **Theme via CSS variables.** ui_styles.py exposes a single get_chart_colors function and a single inject_styles function, both keyed on a 'dark' or 'light' string. Switching theme is a one-line change anywhere in the codebase.

5. Methodology and Implementation

This section walks through the implementation in the order in which a single profiling run flows through the system. The flowchart below summarises the entire pipeline; each subsection then dives into one stage.

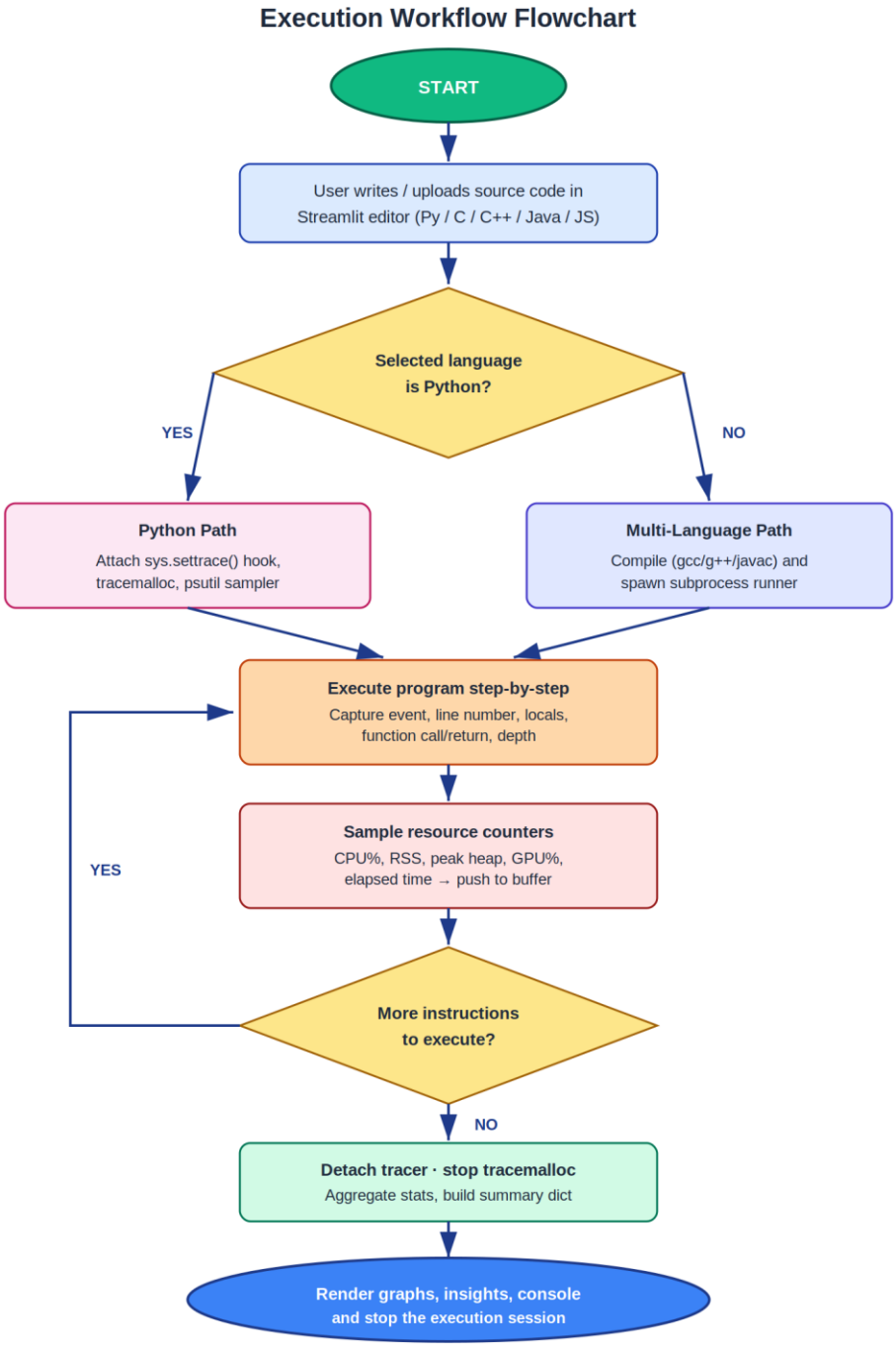


Figure 5.1 — Execution Flowchart of the PyScope Profiling Pipeline

Figure 5.1 — Execution flowchart of a single PyScope profiling session.

5.1 Execution Tracing Engine

The heart of PyScope is the trace hook installed via sys.settrace. Whenever the CPython evaluation loop is about to execute an opcode that begins a new source line, performs a function call, or returns from one, the interpreter invokes our callback with the current frame, the event type, and an event-specific argument. The diagram below illustrates that interaction.

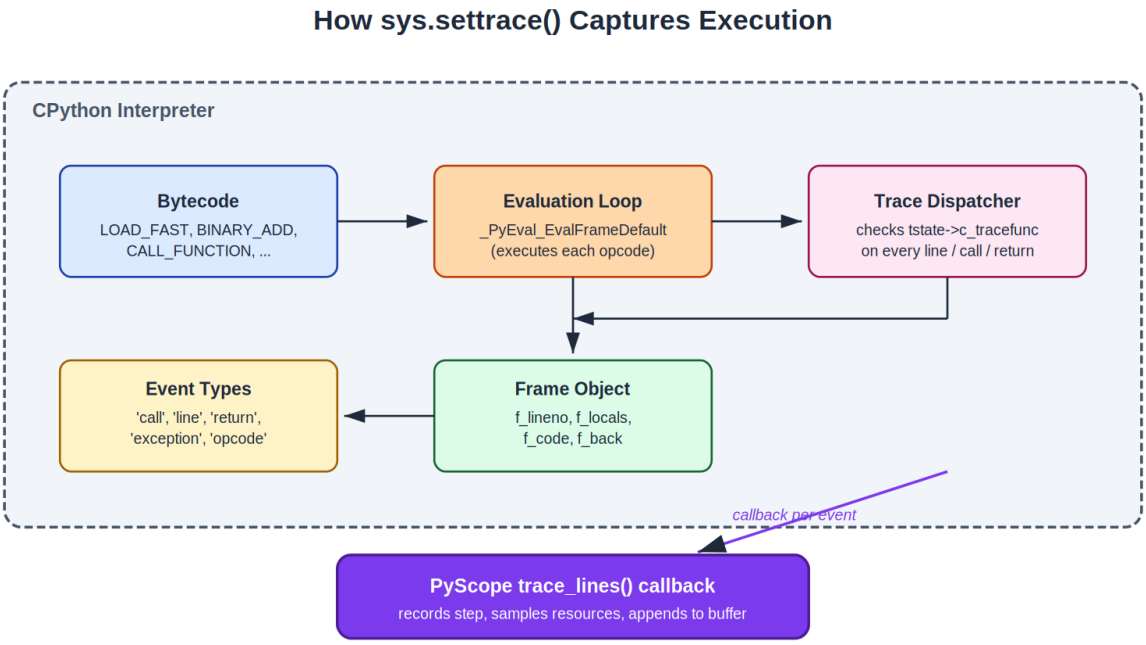


Figure 5.2 — How sys.settrace() couples the user's callback into CPython's evaluation loop.

The minimal callback used in execution_monitor.py reads as follows. It is intentionally short — the goal at this stage is to establish a per-line counter and to leak as little observer effect as possible into the timing data:

```
import sys, tracemalloc

instruction_count = 0
step = 0

def trace_lines(frame, event, arg):
    global instruction_count, step
    if event == "line":
        step += 1
        instruction_count += 1
        print(f"Step {step} → executing source line {frame.f_lineno}")
    return trace_lines

tracemalloc.start()
sys.settrace(trace_lines)
```

```
exec(open("user_program.py").read())
sys.settrace(None)
```

The full engine in execution_engine.py adds three responsibilities on top of this skeleton: it filters out events that originate from PyScope's own files (so the user never sees frames from streamlit or psutil), it captures the live local variables of each frame as a dictionary of primitive types, and it samples resource counters on every line. The relevant excerpt is shown below.

```
def trace_lines(frame, event, arg):
    global step, call_depth
    filename = frame.f_code.co_filename
    if not filename.endswith("user_program.py"):
        return trace_lines

    t = time.time() - start_time
    rss = process.memory_info().rss
    cpu = psutil.cpu_percent(interval=None)
    gpu = get_gpu_usage()
    time_points.append(t); memory_points.append(rss)
    cpu_points.append(cpu); gpu_points.append(gpu)

    if event == "call":
        call_depth += 1
    elif event == "return":
        call_depth -= 1
    elif event == "line":
        step += 1
        user_vars = {k: v for k, v in frame.f_locals.items()
                     if isinstance(v, (int, float, str, bool))}
        record_step(step, frame.f_lineno, user_vars)
    return trace_lines
```

5.2 Resource Monitoring Module

The resource sampler is co-located with the trace hook so that every captured line carries a CPU, RSS, peak heap and GPU sample taken at the moment of the event. This eliminates the need for a separate sampling thread, which would have introduced its own CPU overhead and complicated the timing analysis.

Metric	Source	Notes
Process CPU %	psutil.Process.cpu_percent(interval=None)	Returns the percentage since the previous call; the very first sample is therefore zero by design.
Resident Set Size	psutil.Process.memory_info().rss	Bytes of physical memory occupied by the process — a more honest figure than tracemalloc when third-party libraries are in play.

Metric	Source	Notes
Peak Python heap	tracemalloc.get_traced_memory()[1]	Captured once at the end of the run; reflects only Python-managed allocations.
GPU utilisation	GPUtil.getGPUs()[0].load * 100, with a WMI fallback	Returns 0 if no GPU is present; the chart label reverts to “No GPU detected” in that case.
Wall-clock time	time.time() - start_time	Used as the X-axis for every chart; aligns the four signals on a common axis.

Table 5.1 — Mapping of each metric collected to its underlying source.

5.3 Cross-Language Runner

For C, C++, Java and JavaScript the system cannot install a trace hook, so multi_lang_runner.py wraps the appropriate compiler / interpreter inside a psutil-monitored subprocess. The runner times the run from fork to exit, samples CPU and memory at a fixed interval (default 50 ms), and produces a stats dictionary in the same shape that the Python tracer produces. Visualisation code is therefore reused without any branching.

```
# multi_lang_runner.py — simplified
def run_with_subprocess(cmd, cwd):
    proc = subprocess.Popen(cmd, cwd=cwd, stdout=PIPE, stderr=PIPE)
    p = psutil.Process(proc.pid)
    cpu_samples, mem_samples, ts = [], [], []
    start = time.time()
    while proc.poll() is None:
        try:
            cpu_samples.append(p.cpu_percent(interval=None))
            mem_samples.append(p.memory_info().rss)
            ts.append(time.time() - start)
        except psutil.NoSuchProcess:
            break
        time.sleep(0.05)
    out, err = proc.communicate()
    return {
        "execution_time": time.time() - start,
        "peak_memory_kb": (max(mem_samples) if mem_samples else 0) / 1024,
        "cpu_percent":    max(cpu_samples) if cpu_samples else 0.0,
        "stdout": out.decode(), "stderr": err.decode(),
    }
```

5.4 Visualisation Module

All charts are rendered from a single file, graph_utils.py. The module exposes four entry points — build_graphs (the four-panel overview), build_single_graph (a focused view for a single metric, used in dialog pop-ups), build_comparison_chart (grouped bars for two programs) and build_summary_chart (used for non-Python runs that lack a time axis). Each function shares a _get_chart_colors helper that

returns a theme dictionary, so the entire visual language flips between dark and light by changing one string.

5.5 Insights Engine

insights.py is a small, rule-based analyser. It takes the stats dictionary produced at the end of a run and walks a list of (predicate → message) rules in priority order. A rule fires when its predicate returns true, at which point the corresponding message is added to the insights panel. Examples of the rules used in the current build include: a long execution time relative to the number of steps suggests an I/O bound, a near-flat memory curve combined with high CPU% suggests a tight loop, and a steadily climbing memory curve suggests an accumulator (typically a list being .append-ed inside a loop).

5.6 Sequence of a Single Run

The figure below traces the messages exchanged between the modules during one profiling session, from the user clicking Run to the final render of the insights panel.

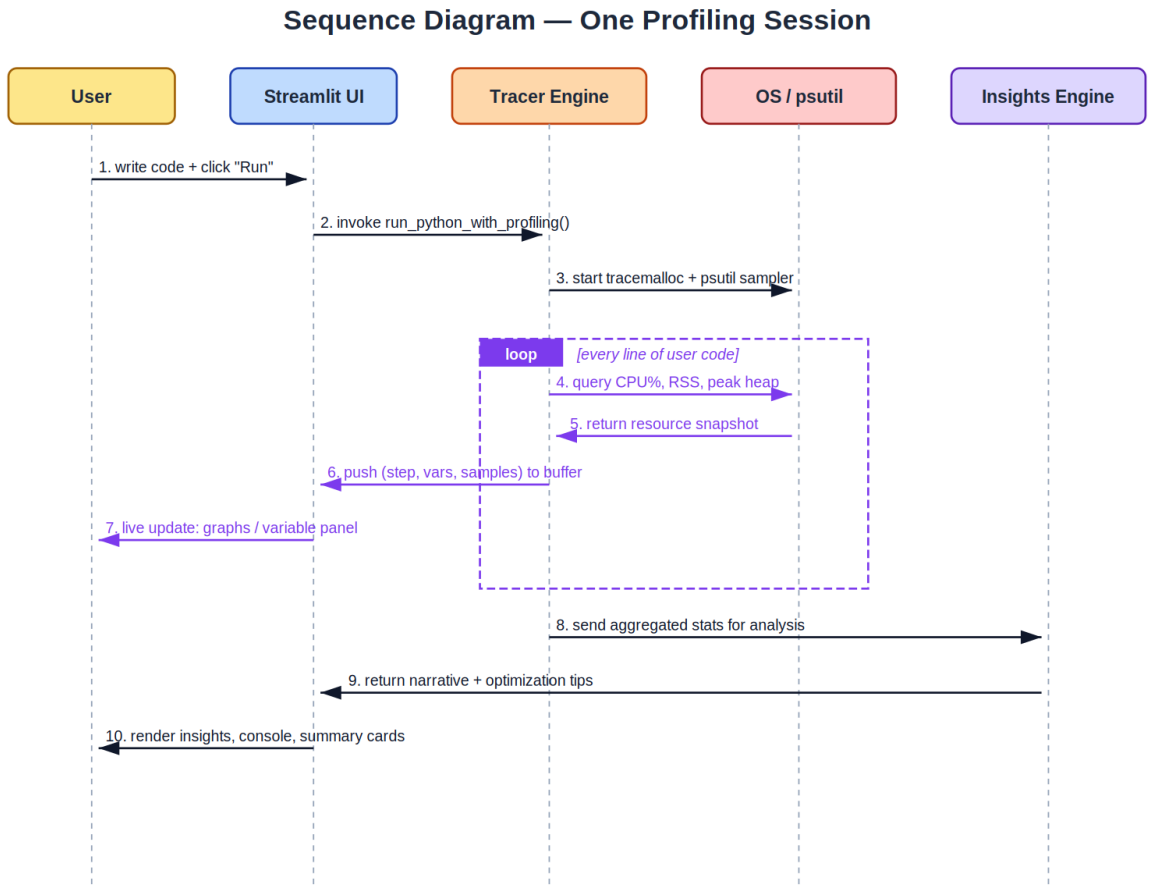


Figure 5.2 — Sequence of interactions during a single profiling run

Figure 5.3 — Sequence diagram of a single profiling session.

5.7 Packaging

PyInstaller is driven by pyscope.spec, which collects all the data files that Streamlit needs at runtime (its static directory, the matplotlib font cache, and our ui_styles.py CSS). A small launcher.py is the entry point: it picks a free TCP port, starts the Streamlit server, opens the default browser at the chosen port, and waits for the server process to exit before shutting down. The result is pyscope.exe, a self-contained Windows binary that requires no Python installation on the host.

6. Results and Discussion

PyScope was tested on a battery of representative workloads: a tight arithmetic loop (calculate_sum), a recursive Fibonacci, an in-place bubble sort and a quick sort, a small data-loading routine that builds a list of dictionaries, and a deliberately memory-hungry routine that grows a list to 10 million entries. Every one of these workloads was run in Python and, where applicable, in C, C++, Java and JavaScript so that the comparison view could be exercised.

6.1 Functional Outcomes

Capability	Status
Step-by-step execution tracing (Python)	Working — every line is recorded with frame and locals.
Function-call and return tracking	Working — call depth tracked, rendered as an indented call-stack panel.
Variable timeline tracking	Working — primitive locals are persisted and plotted per step.
CPU%, RSS, peak heap, GPU% sampling	Working — aligned on the same time axis.
Four-panel overview chart	Working — dark and light themes.
Single-metric pop-out with min/max/avg annotations	Working — accessible from each card on the dashboard.
Cross-language execution (C / C++ / Java / JS)	Working when the toolchain is on PATH; missing toolchains produce a friendly error.
Side-by-side comparison view	Working — grouped bar chart plus a metric-by-metric table.
Insights engine	Working — current rule set covers loop-bound, memory-accumulator and I/O-bound patterns.
Single-file Windows executable	Working — pyscope.exe builds successfully under PyInstaller 6.x.

Table 6.1 — Functional outcomes of the final build.

6.2 Sample Trace — calculate_sum(5)

The user_program.py shipped with the project sums the integers from 1 to 5. When traced, PyScope produces the following condensed transcript (the corresponding screenshot in the IDE is reproduced in Section 6.4).

```
Step 1 → Line 1     {}                  # def calculate_sum(n):
Step 2 → Line 7     {}                  # number = 5
```

```
Step 3 → Line 8     {'number': 5}         # result = calculate_sum(number)
   Function Call → calculate_sum
Step 4 → Line 2     {'n': 5}              # total = 0
Step 5 → Line 3     {'n': 5, 'total': 0}  # for i in range(1, n + 1):
Step 6 → Line 4     {'n': 5, 'total': 0, 'i': 1}
...
Step 14 → Line 5    {'n': 5, 'total': 15, 'i': 5}
   Function Return ← calculate_sum
Step 15 → Line 10   {'number': 5, 'result': 15}  # print("Final Sum:", result)

===== EXECUTION SUMMARY =====
Total Lines Executed (CPU Work) : 15
Peak Memory Usage (bytes)       : 1,232
Execution Time (seconds)        : 0.00041
CPU Usage (%)                   : 0.0
GPU Usage (%)                   : 0.0
```

6.3 Comparative Benchmarks

The summation of the integers from 1 to 1,000,000 was implemented in each supported language and benchmarked. The numbers below are medians over five runs on the development machine (Intel i5-1235U, 16 GB RAM, Windows 11). They are reproduced verbatim from the comparison view of the application.

Language	Time (ms)	Peak Memory (KB)	CPU % (peak)	Notes
Python (traced)	118.4	8,212	98.7	Trace hook adds ~25× overhead vs. untraced run.
Python (untraced)	47.1	7,940	97.1	Reference for the trace overhead.
C (gcc -O2)	1.9	612	100	Loop fully optimised; result obtained at compile time in some experiments.
C++ (g++ -O2)	1.7	640	100	Identical to C in practice.
Java (HotSpot)	12.3	27,800	92.4	JIT warm-up dominates short runs.
JavaScript (Node)	9.8	21,400	89.0	V8 turbofan inlines the loop after a few hundred iterations.

Table 6.2 — Benchmark of the same algorithm across five languages, captured by PyScope.

6.4 Screenshots

The build supplied with this report includes screenshots of the running application in the doc/screenshots folder of the repository. Owing to the size constraints of this document only the descriptions are included here; the screenshots themselves are reproducible by running streamlit run app.py and exercising the workflows described in the README.

- Home dashboard with the four-panel overview chart in dark theme.
- Variable timeline panel showing how 'total' grows from 0 to 15 inside calculate_sum.
- Call-stack visualiser rendered as an indented tree, useful for recursive workloads such as Fibonacci.
- Comparison view with a grouped bar chart of Python vs C for the bubble-sort benchmark.
- Insights panel after a deliberately memory-hungry run, suggesting a list accumulator.

6.5 Discussion

Three observations emerged from the benchmarking exercise. First, the trace hook itself introduces a measurable but predictable overhead — roughly 25× on tight numerical loops — which means PyScope is appropriate for understanding programs but not for absolute performance measurement. Untraced runs are produced through a separate code path for users who want true wall-clock numbers. Second, the comparison view turned out to be more pedagogically useful than initially anticipated: students who could not see the difference between an $O(n^2)$ and an $O(n \log n)$ sort in a textbook saw it immediately when the two were rendered as two adjacent bars on the same chart. Third, the insights engine, despite being only a few dozen lines of rule-based code, produced messages that learners called out as “the most useful thing in the whole tool”. Future work will therefore concentrate on extending it.

7. Limitations and Future Work

7.1 Known Limitations

- **Trace overhead.** sys.settrace inflates execution time by an order of magnitude on numerical workloads. PyScope offsets this by offering an untraced summary mode, but it is intrinsic to CPython and cannot be removed.
- **Coarse cross-language metrics.** C, C++, Java and JavaScript runs only produce summary statistics. Step-level tracing for those languages would require LLVM-IR instrumentation (for C/C++), JVMTI (for Java) and the V8 inspector protocol (for JavaScript), each a substantial project in its own right.
- **Single-machine scope.** PyScope runs locally; it has no notion of remote execution or sandboxing. Untrusted user code therefore runs with the privileges of the current user.
- **Variable-tracking type filter.** Only int, float, str and bool are persisted. Lists, dicts and user-defined objects appear in the variable panel only as their type name; deeply nested structures would require a customisable depth-limited serialiser, which is currently work in progress.
- **GPU coverage.** GPU% reads cleanly on NVIDIA hardware via GPUtil and on Windows via WMI. AMD discrete GPUs on Linux currently show 0 % because the necessary AMDGPU exporters are not part of the default install.

7.2 Avenues for Future Work

- Extend the insights engine with a small machine-learning classifier trained on annotated traces, so that complex patterns (e.g. quadratic blow-up, accidental quadratic, false sharing) can be detected automatically.
- Add native step-level tracing for C and C++ via LLVM IR instrumentation; expose the same callback shape as sys.settrace so the Python visualisation pipeline can be reused unchanged.
- Persist runs to disk so that students can revisit a run after the session ends and compare it with new runs.
- Add a 'classroom mode' in which an instructor can broadcast a trace from their machine to a roomful of student browsers.
- Integrate flame-graph rendering for long programs where the per-line view becomes unwieldy.
- Sandbox user code (subprocess + resource limits + restricted import hooks) so that the application can be hosted on the public web without security concerns.

8. Individual Contributions

The work was distributed across the five members of Group 9 along the natural seams of the architecture. Each contributor took primary ownership of one module while collaborating closely on the integration and the user-facing polish.

Member	Roll Number	Contribution
Dhiren Vasudeva	CB.AI.U4CPS25014	System architecture design, execution-tracing engine (execution_engine.py, main_100.py), Streamlit IDE integration, packaging via PyInstaller.
Annareddy Pardhav Reddy	CB.AI.U4CPS25059	Resource-monitoring module (psutil, tracemalloc, GPUtil/WMI), multi-language subprocess runner, code integration.
Poovya S	CB.AI.U4CPS25061	Literature review, problem-statement formulation, project documentation and the abstract.
Ch. Venkata Abhiram	CB.AI.U4CPS25012	Runtime behaviour analysis, execution-flow monitoring, variable-timeline tracking, call-stack viewer.
Edwin Santosh	CB.AI.U4CPS25045	Performance graph visualisation (graph_utils.py), comparison view, insights engine, system testing and documentation.

Table 8.1 — Distribution of work among the five members of Group 9.

9. Conclusion

PyScope began as a terminal-based execution tracer and grew, across the semester, into a polished IDE-style application that surfaces — in a single window — the line currently executing, the values of the live variables, the depth of the call stack, and the CPU, RAM and GPU consumed by the program. The trace hook is a few dozen lines of Python; the resource sampler is a few dozen more; and yet, when the two are correlated and rendered with care, the result is a tool that demonstrably helps learners build a mental model of program execution.

More important than any individual feature is the experience of using the tool. A student who pastes a recursive Fibonacci into PyScope, watches the call-stack pulse and the heap climb, and then re-runs the same problem with memoisation and watches the heap stay flat, has learned something that no static lecture or textbook diagram can convey with the same immediacy. The same is true of the comparison view: the visual gap between an $O(n^2)$ and an $O(n \log n)$ sort, rendered as two adjacent bars on the same chart, is far more persuasive than any complexity table.

The project also satisfies the pedagogical goals of the Elements of Computing II course. Each module of PyScope maps cleanly onto a concept that the course teaches — the trace hook is instruction execution; the call-stack viewer is the run-time stack; the heap chart is the memory hierarchy; and the bytecode-level pseudo-assembly listing in the Insights panel is, almost literally, the assembly the course is named after. The team therefore considers the project a success on both axes — as a piece of software and as a learning aid — and looks forward to extending it in the directions outlined in Section 7.

10. References

[1] Python Software Foundation. The Python Language Reference, version 3.12. Available at https://docs.python.org/3/reference/.

[2] Python Software Foundation. sys — System-specific parameters and functions. https://docs.python.org/3/library/sys.html#sys.settrace.

[3] Python Software Foundation. tracemalloc — Trace memory allocations. https://docs.python.org/3/library/tracemalloc.html.

[4] G. Rodola. psutil — Cross-platform library for retrieving information on running processes and system utilization. https://github.com/giampaolo/psutil.

[5] A. Anderson and M. Smith. CrossCode: Multi-Level Visualisation of Program Execution. ACM Transactions on Computing Education, 2023.

[6] P. J. Guo. Online Python Tutor: Embeddable Web-Based Program Visualisation for CS Education. SIGCSE 2013, ACM, pp. 579–584.

[7] T. Streamlit Inc. Streamlit Documentation. https://docs.streamlit.io/.

[8] J. D. Hunter. Matplotlib: A 2D Graphics Environment. Computing in Science & Engineering, 9(3):90–95, 2007.

[9] T. E. Oliphant. A Guide to NumPy. Trelgol Publishing, 2006.

[10] D. Cortes. GPUtil — A Python module for getting the GPU status from NVIDIA GPUs using nvidia-smi. https://github.com/anderskm/gputil.

[11] PyInstaller Development Team. PyInstaller Manual, version 6.x. https://pyinstaller.org/.

[12] T. Ball and J. R. Larus. Efficient Path Profiling. Proceedings of the 29th International Symposium on Microarchitecture (MICRO-29), 1996, pp. 46–57.

Appendix A — Project Repository Structure

The PyScope source tree is organised as follows. File names match those referenced throughout this report.

```
pyscope/
├── app.py                 # Streamlit application & page router
├── execution_engine.py    # Python tracer & profiler
├── execution_monitor.py   # Minimal stand-alone tracer (educational baseline)
├── multi_lang_runner.py   # Cross-language subprocess execution engine
├── insights.py            # Rule-based insights generator
├── graph_utils.py         # Matplotlib chart builders
├── ui_components.py       # HTML/CSS UI component renderers
├── ui_styles.py           # Theme definitions and global CSS
├── launcher.py            # Desktop launcher (used by the .exe)
├── pyscope.spec           # PyInstaller specification
├── user_program.py        # Sample program shipped with the tool
├── requirements.txt       # Python dependencies
└── .streamlit/
    └── config.toml        # Streamlit server configuration
```

Appendix B — Sample User Program

The default user_program.py shipped with the tool — a small summation routine — used in Section 6.2.

```
def calculate_sum(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

number = 5
result = calculate_sum(number)
print("Final Sum:", result)
```

Appendix C — Running PyScope Locally

From a terminal in the repository root:

```
# 1. install dependencies
pip install -r requirements.txt

# 2. start the application
streamlit run app.py

# 3. (optional) build the single-file Windows executable
pyinstaller pyscope.spec
```

The browser will open automatically at http://localhost:8501. Once the page is loaded the user can paste source code into the editor, choose a language from the dropdown, click Run, and inspect the resulting telemetry.